

Table 6.1: Sprint-wise Development Plan

Sprint	Focus Area	Deliverables	Duration
Sprint 1	Project Setup	Next.js project initialization, Firebase configuration, authentication setup, role-based routing	2 weeks
Sprint 2	Core Features	Waste listing CRUD operations, Firestore data models, listing status workflow (OPEN → ASSIGNED → IN_TRANSIT → DELIVERED)	2 weeks
Sprint 3	Dashboard UIs	Farmer dashboard, Agent dashboard, Admin dashboard with role-specific components and statistics cards	2 weeks
Sprint 4	Integrations	Razorpay payment gateway, Cloudinary photo uploads, GPS location services, notification system	2 weeks
Sprint 5	Advanced Features	Carbon credit tracking, rewards/gamification system, AI waste classification via Genkit, CSV export	2 weeks
Sprint 6	Polish & Deploy	Responsive design refinements, performance optimization, security rules, accessibility improvements, deployment	2 weeks

- **Application Layer:** Next.js API routes for server-side processing (Razorpay webhooks, admin operations), Firebase service functions for data operations, and custom React hooks for state management.
- **Data Layer:** Firebase Firestore for persistent NoSQL storage, Firebase Authentication for identity management, and Firebase Storage for media files (waste photos, delivery proofs).

6.4 Development Tools and Environment

The following tools were used throughout the development process:

6.5 Data Flow

The data flow in AgriTrace follows a unidirectional pattern:

1. Farmer creates a waste listing with details (type, quantity, location, photos).
2. Listing is stored in Firestore and becomes visible to agents.
3. Agent claims the listing and initiates pickup.
4. Agent marks the waste as “In Transit” and later as “Delivered”.